

CodeCurry — DSA Feedback Agent

1. Project Overview

CodeCurry is an AI-powered DSA learning and assessment platform that helps students practice LeetCode-style problems and receive structured feedback on their solutions.

The system contains around **20 Simple and Medium DSA problems**. Students can solve problems, submit their code, and receive a rubric-based evaluation including correctness, time complexity, space complexity, code quality, edge-case handling, and improvement suggestions.

The system uses **Python, Streamlit, PostgreSQL, pgvector, RAG, LLM, Memory, ReAct, and one Rubric Generator Agent**.

2. Project Objectives

The main objectives of CodeCurry are:

1. Provide Simple and Medium DSA problems for practice.
 2. Store and retrieve DSA knowledge using RAG.
 3. Ingest and chunk problem-related information.
 4. Generate a problem-specific evaluation rubric.
 5. Evaluate submitted solutions against the rubric.
 6. Provide score, strengths, weaknesses, and suggestions.
 7. Remember previous attempts and feedback.
 8. Use ReAct-based reasoning for the Rubric Agent.
 9. Provide a simple interactive interface using Streamlit.
-

3. Key Features

DSA Problem Practice

Students can:

- Select a DSA problem.
- View description, constraints, and examples.
- Submit their solution.
- View evaluation and feedback.

AI DSA Feedback

The system evaluates:

- Correctness
- Time Complexity
- Space Complexity
- Code Quality
- Edge Cases
- Overall Approach

Persistent Memory

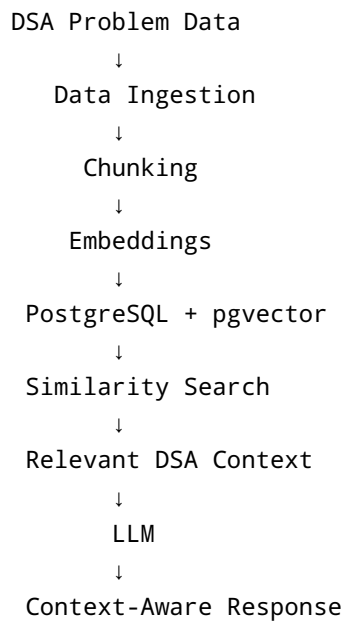
The system stores:

- Previous submissions
- Scores
- Feedback
- Conversation history
- Search history

This allows CodeCurry to provide more personalized feedback.

4. RAG Workflow

CodeCurry uses **Retrieval-Augmented Generation (RAG)** to provide problem-specific context to the LLM.



For example, when evaluating **Two Sum**, the system can retrieve its description, expected approach, complexity, and rubric instead of sending the entire dataset to the LLM.

5. Rubric Generator Agent

CodeCurry contains **ONLY ONE AGENT**:

Rubric Generator Agent

The agent creates a problem-specific rubric and uses it to evaluate the student's submitted solution.

Example:

Two Sum Rubric

Criteria	Weight
Correctness	40%
Time Complexity	20%
Space Complexity	15%
Code Quality	15%
Edge Cases	10%
Total	100%

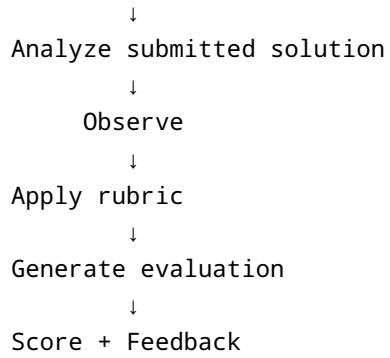
The rubric may change depending on the problem and difficulty.

6. ReAct Loop

The Rubric Generator Agent follows a **Reason → Act → Observe** workflow.

Example:

```
Student submits code
  ↓
Reason
  ↓
Retrieve problem + rubric
  ↓
Observe
```



The ReAct process is handled internally by the single Rubric Generator Agent.

7. Solution Evaluation

Example student submission:

Problem: Two Sum

Student Solution:
Nested-loop approach

Test Result:
All test cases passed

Analysis:
Time Complexity → $O(n^2)$
Space Complexity → $O(1)$

Example output:

Overall Score: 82/100

Correctness: 36/40
Time Complexity: 16/20
Space Complexity: 12/15
Code Quality: 11/15
Edge Cases: 7/10

Strengths:

- Correct solution
- Handles normal test cases
- Simple implementation

Suggestions:

- Improve time complexity
- Consider a HashMap approach
- Handle edge cases explicitly

8. Memory

Memory allows CodeCurry to remember previous interactions.

Example:

Student History

Two Sum

- └─ Attempt 1 → 65/100
- └─ Attempt 2 → 74/100
- └─ Attempt 3 → 82/100

The system can identify repeated mistakes and provide personalized suggestions.

9. PostgreSQL + pgvector

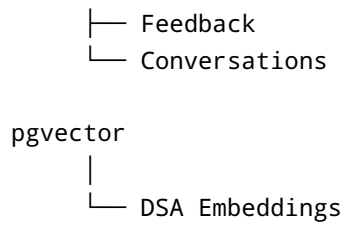
PostgreSQL will store persistent application data such as:

- Problems
- Conversations
- Messages
- Submissions
- Scores
- Feedback
- Search history

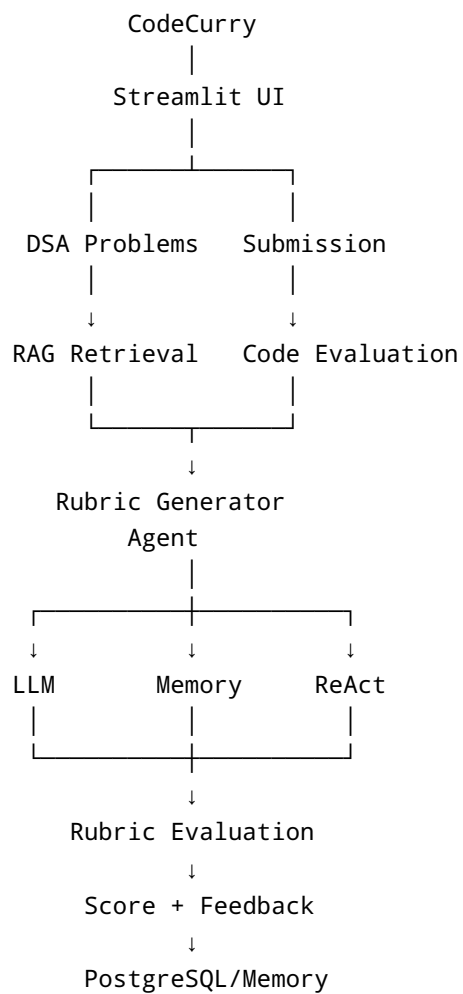
pgvector will store embeddings and support similarity search for RAG.

PostgreSQL

- └─ Problems
- └─ Users
- └─ Submissions



10. System Architecture



There is only one agent in this architecture: the Rubric Generator Agent.

11. Implementation Process

Step 1 — Create Dataset

Prepare approximately 20 Simple and Medium DSA problems.

Step 2 — Data Ingestion

Load the problem data into PostgreSQL/knowledge storage.

Step 3 — Chunking

Divide problem information into meaningful chunks such as description, constraints, approach, complexity, and rubric.

Step 4 — Embeddings + pgvector

Generate embeddings for chunks and store them in pgvector.

Step 5 — Retrieval

Retrieve relevant chunks when a problem or solution needs evaluation.

Step 6 — Rubric Generator

Generate the evaluation rubric for the selected problem.

Step 7 — Code Evaluation

Run the student's solution against predefined test cases and analyze its complexity.

Step 8 — Rubric Agent

The single Rubric Generator Agent combines:

```
Problem Context
+
Student Code
+
Test Results
+
Rubric
+
Memory
```

and produces the evaluation.

Step 9 — ReAct + LLM

The agent uses the ReAct workflow and LLM to reason over the available information and generate structured feedback.

Step 10 — Store Memory

Save the score, mistakes, submission, and feedback for future attempts.

12. Project Structure

```
CodeCurry/
├── app.py
├── requirements.txt
├── README.md
├── agents/
│   └── rubric_agent.py
├── rag/
│   ├── ingestion.py
│   ├── chunking.py
│   └── retrieval.py
├── database/
│   ├── connection.py
│   └── schema.sql
├── memory/
│   └── memory.py
├── evaluation/
│   ├── code_runner.py
│   └── complexity.py
└── data/
    ├── problems/
    │   ├── two_sum.json
    │   └── binary_search.json
```

```
|— valid_parentheses.json
|— ...
```

13. Final Workflow

```
User
↓
Select DSA Problem
↓
Submit Solution
↓
Orchestrator
↓
Retrieve Relevant Context
↓
Run Test Cases
↓
Load Rubric + Memory
↓
ONE RUBRIC GENERATOR AGENT
↓
ReAct + LLM
↓
Evaluate Solution
↓
Score + Feedback
↓
Save Memory
↓
Display Result
```

14. Task 1 Deliverable

Create and push:

```
docs/
|— SCOPE_AND_IMPLEMENTATION.md
```

This document defines the initial scope, architecture, technology components, single-agent design, RAG workflow, and implementation steps for CodeCurry.